



ARQUITETURA DO SISTEMA

Guia técnico para quem está começando

Frontend React · API NestJS/Fastify · Supabase PostgreSQL · BigQuery

1. O que estamos construindo?

Uma visão simples antes de entrar nos detalhes técnicos.

O Glazia é um SaaS de controle financeiro para vidraçarias e empresas de esquadrias. “SaaS” significa que várias empresas usam o mesmo produto pela internet, mas cada uma enxerga somente os próprios dados.

A regra mais importante do sistema

Nenhuma empresa pode visualizar, alterar ou inferir dados de outra. Chamamos isso de isolamento multi-tenant. O campo `id_empresa` acompanha todos os dados de negócio e a segurança é aplicada em mais de uma camada.

O sistema possui quatro grandes partes

Frontend

Telas usadas pelo cliente. React, Vite e TypeScript.

API

Guarda-costas que valida o usuário e executa regras.

Supabase

Login e dados operacionais do dia a dia.

BigQuery

Relatórios, histórico e análises pesadas.

O frontend nunca acessa o BigQuery. A API valida o JWT, descobre a empresa e faz a consulta.

Estado atual da implementação

- Frontend visual do Glazia em React, responsivo e com modo claro/escuro.
- API NestJS executando com o adaptador Fastify.
- Login integrado ao Supabase Auth.
- CRUD inicial de lançamentos financeiros conectado à API.
- RLS, índices e validações multi-tenant aplicados no Supabase.
- BigQuery modelado, mas a sincronização ETL ainda é a próxima fase.

2. Glossário essencial

Os termos que aparecem com frequência no projeto.

Termo	Significado simples
Frontend	A parte visual que roda no navegador do usuário.
Backend / API	O servidor que valida pedidos, aplica regras e acessa dados.
JWT	Um token assinado que comprova quem fez login.
Tenant	Uma empresa cliente dentro do SaaS.
Multi-tenant	Várias empresas usam a mesma aplicação com dados isolados.
RLS	Regras do PostgreSQL que filtram linhas conforme o usuário.
OLTP	Banco voltado para inserts, updates e operação diária.
OLAP	Banco voltado para grandes análises e agregações.
CQRS	Separação entre o caminho operacional e o analítico.
ETL / ELT	Processo de copiar e transformar dados entre bancos.
Schema estrela	Modelo analítico com fatos no centro e dimensões ao redor.
Idempotência	Executar novamente sem duplicar ou corromper dados.

Uma analogia útil

Pense no Supabase como o caixa e o estoque da loja: registra cada operação imediatamente. O BigQuery é a sala de análise: recebe cópias organizadas para responder perguntas históricas e estratégicas.

3. Como o login protege cada empresa

O caminho completo desde a senha até uma consulta financeira.

- 1 Login no Supabase Auth**
O frontend envia e-mail e senha ao Supabase. A senha não passa pela API do Glazia.
- 2 Supabase devolve o JWT**
O token representa a sessão autenticada. Ele possui validade e assinatura verificável.
- 3 Frontend chama a API**
Cada chamada envia Authorization: Bearer <token>.
- 4 AuthGuard valida a sessão**
A API pede ao Supabase a confirmação de que o token ainda representa um usuário válido.
- 5 API resolve a empresa**
O usuário é localizado em perfis pelo auth.uid(). O id_empresa não é aceito do navegador.
- 6 Cliente Supabase recebe o JWT**
As consultas da API são executadas com o token do usuário, mantendo o RLS ativo.
- 7 Filtro explícito é aplicado**
Além do RLS, cada query de negócio inclui o id_empresa do contexto seguro.

Por que duas camadas?

A API filtra para evitar erros de programação. O RLS filtra novamente dentro do banco. Se uma camada falhar, a outra continua protegendo os dados. Isso é defesa em profundidade.

Exemplo conceitual

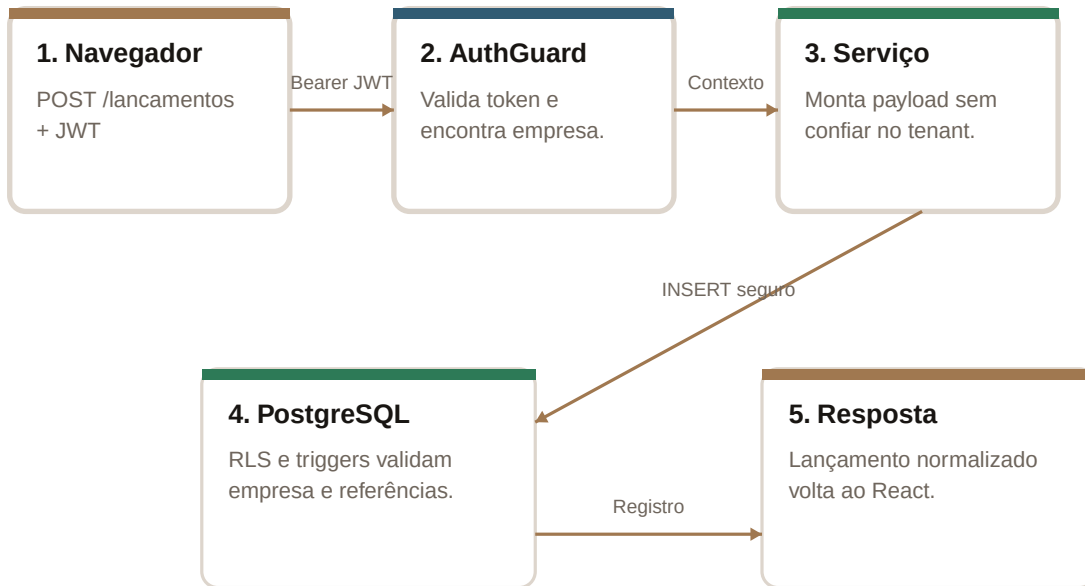
```
JWT -> AuthGuard -> perfil do usuário -> empresaId seguro
```

```
SELECT * FROM lancamentos_financeiros  
WHERE id_empresa = empresaIdSeguro;
```

O frontend nunca decide qual empresa consultar. Um campo id_empresa enviado pelo navegador é ignorado e rejeitado pela validação dos DTOs.

4. Fluxo de uma requisição

Exemplo: registrar uma entrada financeira.



O que acontece se houver fraude ou erro?

- JWT ausente ou inválido: a API responde HTTP 401.
- Usuário sem perfil/empresa: a API responde HTTP 401.
- ID de cliente de outra empresa: trigger do banco bloqueia a operação.
- Campo não permitido no body: ValidationPipe rejeita a requisição.
- Registro inexistente: a API responde HTTP 404 quando aplicável.

5. Banco operacional: Supabase/PostgreSQL

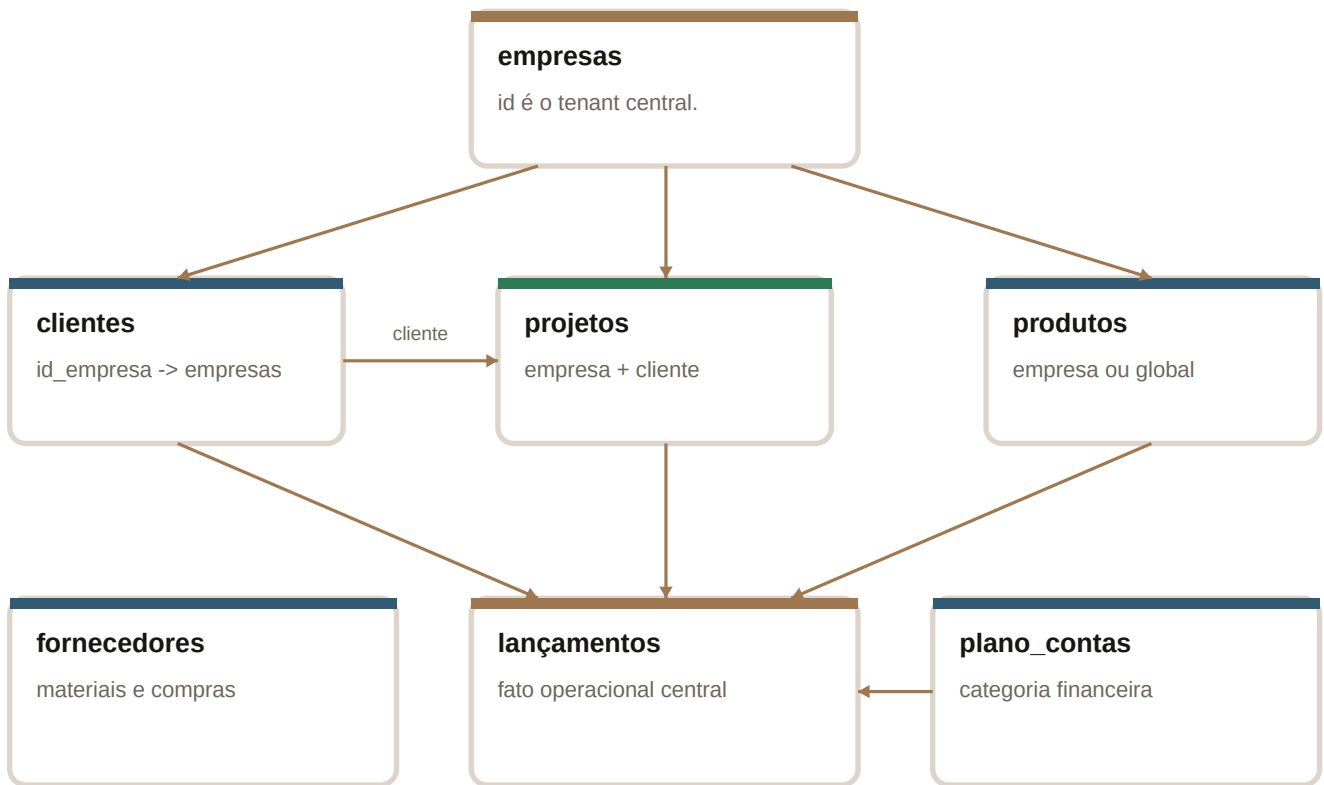
Fonte da verdade para a operação diária.

O PostgreSQL guarda dados que mudam constantemente: usuários, clientes, fornecedores, projetos, produtos, lançamentos e despesas fixas. Essas tabelas são o OLTP do Glazia.

Tabela	Responsabilidade	Ligação principal
empresas	Representa cada tenant.	Raiz do isolamento.
perfis	Liga auth.users à empresa.	id = auth.uid().
clientes	Clientes da vidraçaria.	Pertence à empresa.
fornecedores	Fornecedores de materiais.	Pertence à empresa.
produtos	Produtos finais e insumos.	Empresa ou catálogo global.
projetos	Obras vinculadas a clientes.	Empresa + cliente.
plano_contas	Classificação financeira.	Empresa ou catálogo global.
lançamentos_financeiros	Entradas, saídas e previsões.	Relaciona os cadastros.
despesas_fixas	Compromissos recorrentes.	Empresa + plano de contas.

6. Relações entre as tabelas

Como ler o modelo relacional.



Integridade entre tenants

Uma foreign key comum confirma que um ID existe, mas não garante que ele pertence à mesma empresa. Por isso o Glazia possui triggers privados que validam a combinação entre o registro referenciado e id_empresa.

Exemplo bloqueado

A Empresa B tenta criar um projeto usando um cliente da Empresa A. O cliente existe, mas a trigger validar_projeto_tenant detecta a divergência e cancela o INSERT.

7. Como o RLS funciona

Segurança aplicada dentro do PostgreSQL.

RLS significa Row-Level Security, ou segurança por linha. Ao consultar uma tabela, o PostgreSQL avalia políticas antes de devolver cada registro.

Função de contexto

```
private.empresa_logada()  
  -> lê auth.uid()  
  -> procura o usuário em public.perfis  
  -> devolve o id_empresa vinculado
```

A função fica no schema private, possui search_path fixo e não pode ser chamada anonimamente. Ela é SECURITY DEFINER porque precisa encontrar o perfil sem entrar em recursão com a própria política de perfis.

Tipos de política

- SELECT: permite visualizar somente registros da empresa autenticada.
- INSERT: exige que o novo registro use a empresa autenticada.
- UPDATE: valida tanto o registro atual quanto o resultado atualizado.
- DELETE: permite remover apenas registros pertencentes ao tenant.
- Produtos e planos globais: podem ser lidos, mas não alterados por empresas.

Atenção com service_role

A chave service_role ignora RLS. Ela existe apenas para tarefas administrativas controladas, como o bootstrap do primeiro tenant. Nunca deve chegar ao React, ao navegador, a logs ou ao repositório Git.

8. API NestJS + Fastify

A fronteira segura entre o navegador e os dados.

NestJS organiza o backend em módulos, controllers, services, guards e DTOs. Fastify é o servidor HTTP usado por baixo, escolhido por ser rápido e leve.

Rota	Uso	Proteção
GET /api/v1/health	Verifica se a API está no ar.	Pública
GET /api/v1/me	Usuário e empresa autenticada.	JWT
GET /api/v1/catalogos	Clientes, produtos, projetos e contas.	JWT + tenant
GET /api/v1/lancamentos	Lista lançamentos da empresa.	JWT + tenant
POST /api/v1/lancamentos	Cria entrada ou saída.	JWT + DTO + RLS
DELETE /api/v1/lancamentos/:id	Exclui lançamento da empresa.	JWT + tenant

Responsabilidade de cada

- Controller: recebe HTTP e encaminha para o service.
- DTO: valida formato, tipos e campos permitidos.
- AuthGuard: valida JWT e monta o contexto seguro.
- Service: executa a regra de negócio e as queries.
- SupabaseService: cria clientes Supabase com a identidade correta.
- ConfigModule: impede a API de iniciar com variáveis inválidas.

Regra de implementação

Nunca aceite id_empresa do cliente

O DTO não possui id_empresa. O service recebe empresald separadamente, vindo do AuthGuard. Essa separação torna difícil misturar dados por acidente.

9. Integração com o frontend

Como o protótipo passou a usar dados reais.

Antes

- Login aceitava qualquer senha.
- Lançamentos vinham de arrays em seed.ts.
- Alterações eram salvas no localStorage.

Agora

- Login usa signInWithPassword do Supabase Auth.
- A sessão é persistida e renovada pelo cliente Supabase.
- A camada src/services/api.ts envia o JWT para a API.
- O AppContext carrega usuário, catálogos e lançamentos reais.
- O formulário cria clientes por nome quando necessário.
- Despesas e dashboards ainda permanecem parcialmente simulados nesta fase.

Por que integrar por etapas?

Trocar tudo de uma vez aumenta o risco. A fatia vertical de login + lançamentos prova autenticação, API, RLS e frontend. Depois repetimos o padrão para despesas, projetos e dashboards.

Arquivos principais

src/services/supabase.ts	-> sessão e cliente Supabase
src/services/api.ts	-> chamadas HTTP autenticadas
src/context/AppContext.tsx	-> estado e integração das telas
src/pages/Login.tsx	-> login real
src/pages/Lancamentos.tsx	-> CRUD conectado

10. Banco analítico: BigQuery

Onde ficam relatórios e análises históricas.

O BigQuery não substitui o PostgreSQL. Ele recebe cópias transformadas dos dados para responder consultas grandes com eficiência. O dataset analytics usa um esquema estrela.

Dimensões: quem, o quê e onde

- dim_empresa: identifica a empresa.
- dim_cliente: descreve o cliente.
- dim_produto: produto, linha e unidade.
- dim_projeto: obra, cliente e status.
- dim_plano_contas: categoria e subcategoria.
- dim_fornecedor: fornecedor e prazo médio.

Fatos: eventos mensuráveis

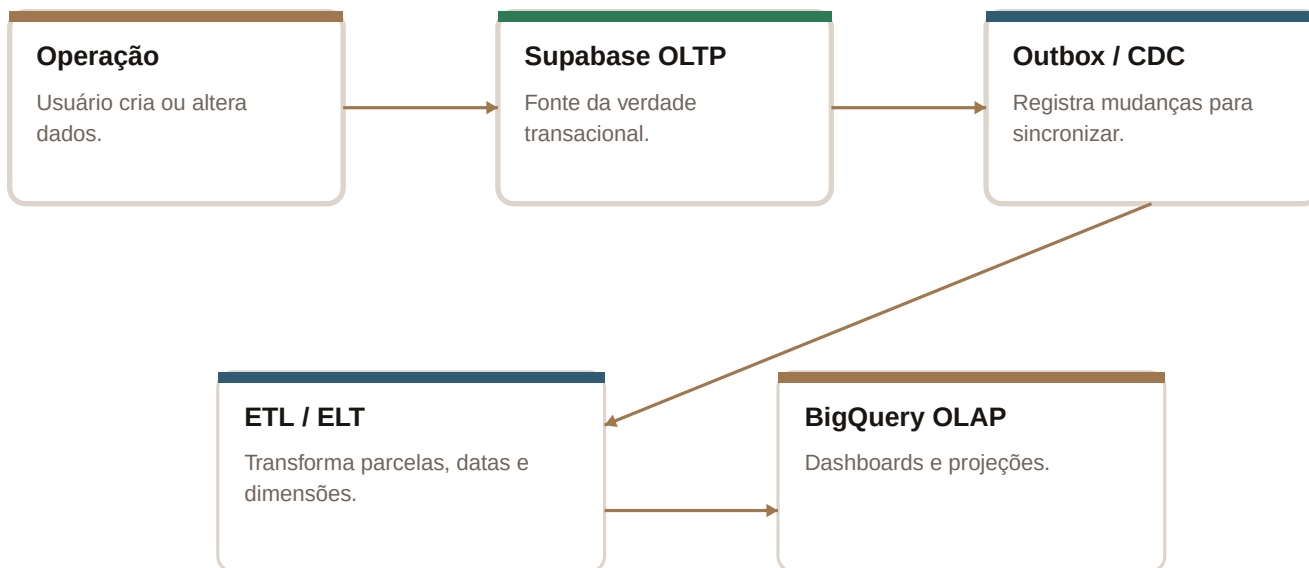
- fato_vendas: valor vendido e custo estimado por projeto/produto.
- fato_custos_operacionais: materiais, quantidades e fornecedores.
- fato_fluxo_caixa: previsto, realizado, vencimento e pagamento.

Performance e custo

As fatos devem ser particionadas pela coluna de data e clusterizadas por id_empresa. A API analítica também deve exigir filtro de tenant e, quando possível, filtro de partição.

11. CQRS e sincronização

Como Supabase e BigQuery trabalham juntos.



Consistência eventual

Um lançamento aparece imediatamente no Supabase, ao BigQuery. Isso é esperado. Telas operacionais devem refletir o estado do BigQuery.

Requisitos do pipeline

- Idempotência: reprocessar sem duplicar.
- Checkpoint: saber até onde os dados foram processados.
- Auditoria: registrar falhas e quantidades carregadas.
- Dead letter: separar eventos problemáticos para análise.
- Observabilidade: métricas, logs e alertas de atraso.

12. Como executar localmente

Passos para desenvolvimento no Windows.

1. Configurar o frontend

```
copie .env.example para .env.local  
preencha VITE_SUPABASE_PUBLISHABLE_KEY
```

2. Configurar a API

```
copie server/.env.example para server/.env  
preencha SUPABASE_PUBLISHABLE_KEY
```

3. Criar a primeira empresa e usuário

O bootstrap é uma operação administrativa. Coloque temporariamente a SUPABASE_SERVICE_ROLE_KEY somente em server/.env, defina e-mail, senha e nome da empresa e execute:

```
cd server  
npm run bootstrap:tenant
```

Depois do bootstrap, remova a service_role do ambiente local se ela não for mais necessária.

4. Iniciar tudo

```
npm run dev:full
```

Frontend: <http://localhost:5173>

Saúde da API: <http://localhost:3000/api/v1/health>

13. Checklist de segurança

Use esta lista em toda funcionalidade nova.

- ☐ 1. A rota exige JWT, exceto se for explicitamente pública?
- ☐ 2. O id_empresa vem do contexto autenticado?
- ☐ 3. O DTO rejeita id_empresa enviado pelo navegador?
- ☐ 4. Toda query de negócio possui filtro explícito de tenant?
- ☐ 5. O RLS cobre SELECT, INSERT, UPDATE e DELETE?
- ☐ 6. Referências a cliente/projeto/produto pertencem ao tenant?
- ☐ 7. A service_role está apenas no backend administrativo?
- ☐ 8. Logs evitam JWT, senhas, chaves e dados financeiros sensíveis?
- ☐ 9. Testes usam ao menos duas empresas?
- ☐ 10. Exports, cache, IA e BigQuery também carregam id_empresa?

Princípio central

Segurança multi-tenant não é apenas um WHERE. Ela precisa existir no contexto da API, nas políticas RLS, na integridade relacional, nos testes e também na camada analítica.

14. Próximos passos recomendados

Ordem segura para continuar a evolução.

1

Bootstrap do primeiro tenant

Criar usuário, empresa e catálogos iniciais com o script administrativo.

2

Despesas fixas reais

Criar módulo da API e substituir o localStorage da tela correspondente.

3

Cadastros completos

CRUD de clientes, fornecedores, produtos e projetos.

4

Dashboard operacional

Indicadores recentes calculados no PostgreSQL enquanto o ETL não está pronto.

5

Outbox e sincronização

Implementar pipeline confiável do Supabase para o BigQuery.

6

Dashboard analítico

Migrar agregações históricas para consultas parametrizadas no BigQuery.

7

Glazia IA

Responder com dados reais, sempre usando o contexto seguro de tenant.

8

Observabilidade

Adicionar logs estruturados, métricas, rastreamento e alertas.

Definição de pronto para cada módulo

- Código compilando e lint sem erros.
- Testes de sucesso, erro e isolamento entre duas empresas.
- RLS e advisors verificados após alterações no banco.
- Nenhuma credencial inserida no Git ou enviada ao navegador.
- Contrato da API documentado e consumido pelo frontend.

15. Mapa do repositório

Onde procurar quando precisar alterar algo.

Caminho	O que contém
src/pages	Telas React do produto.
src/components	Layout, loading e chat flutuante.
src/context	Sessão e estado compartilhado.
src/services	Cliente Supabase e chamadas da API.
server/src/auth	Guard, contexto e decorators de autenticação.
server/src/catalogos	Leitura dos cadastros auxiliares.
server/src/lancamentos	DTO, controller, service e testes.
server/src/supabase	Criação dos clientes Supabase.
server/scripts	Bootstrap administrativo do primeiro tenant.
supabase/migrations	Histórico versionado do banco e RLS.
docs	Documentação gerada do projeto.

Como raciocinar ao receber uma tarefa

- 1 Identifique o domínio**
É login, lançamento, projeto, análise ou sincronização?
- 2 Defina a fonte**
Operação diária lê Supabase; histórico pesado lê BigQuery.
- 3 Aplique o tenant**
Resolva id_empresa pelo JWT e filtre todas as camadas.
- 4 Implemente verticalmente**
Banco, API, frontend e teste do menor fluxo completo.
- 5 Teste a fronteira**
Tente acessar dados com um segundo tenant e confirme o bloqueio.

Resumo final

O Glazia separa operação, segurança e análise para crescer sem misturar os dados de seus clientes.

Supabase

Autenticação e fonte transacional. Recebe as operações do dia a dia e aplica RLS.

API NestJS

Valida o JWT, resolve a empresa e centraliza as regras de negócio.

BigQuery

Recebe dados transformados para relatórios, projeções e análises históricas.

Frontend React

Apresenta a experiência do usuário e nunca decide sozinho qual empresa consultar.

Se você lembrar de uma única regra, lembre desta: toda funcionalidade deve ser projetada como se já existissem muitas empresas, mesmo quando o sistema ainda possui apenas um cliente.